

Experiment 2: Explore Git and GitHub Commands

Tools Required: Git, GitHub Account, Git Bash, Visual Studio Code (VS Code)

1. Aim

To understand and implement **Git Version Control System** and **GitHub** by performing repository creation, tracking files, committing changes, pushing code to GitHub, and cloning repositories using Git Bash and VS Code.

2. Objectives

After completing this experiment, students will be able to:

- Understand Version Control System (VCS)
 - Install Git and configure it
 - Create GitHub account
 - Create local repository
 - Create remote repository on GitHub
 - Connect local repository with GitHub
 - Perform Git commands
 - Push and Pull code
 - Clone repositories
 - Use Git inside VS Code
-

3. Theory

What is Version Control?

Version Control is a system that records changes made to files over time.

It allows us to:

- Save different versions of project

- Track every modification
- Restore previous version
- Work with team members
- Avoid overwriting files

Example

Suppose you are writing a Java program.

Without Version Control

```
Program.java  
Program_new.java  
Program_latest.java  
Program_final.java  
Program_final2.java  
Program_final_final.java
```

Very confusing.

With Git

```
Version 1  
Version 2  
Version 3  
Version 4
```

Git stores every change.

Why Git?

Git is a **Distributed Version Control System (DVCS)**.

Features

- Fast
 - Free
 - Open Source
 - Tracks changes
 - Supports collaboration
 - Used by companies worldwide
-

What is GitHub?

GitHub is an online platform that stores Git repositories.

Git → Local machine

GitHub → Cloud storage

Example

```
Laptop
|
Git Repository
|
Push
|
GitHub Repository
```

Difference Between Git and GitHub

Git	GitHub
Software	Online Platform
Works locally	Works on cloud
Tracks changes	Stores repositories
Command line tool	Web application
No Internet required	Internet required

4. Software Required

- Windows 10/11
 - Git
 - GitHub Account
 - Git Bash
 - VS Code
-

5. Installation

Step 1 Install Git

Download Git.

Install with default settings.

Verify Installation

Open Command Prompt

```
git --version
```

Output

```
git version 2.xx.x
```

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt". The window content shows the following text: "Microsoft Windows [Version 10.0.19045.6466]", "(c) Microsoft Corporation. All rights reserved.", "C:\Users\Admin>git --version", "git version 2.55.0.windows.2", and "C:\Users\Admin>". The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin>git --version
git version 2.55.0.windows.2

C:\Users\Admin>
```

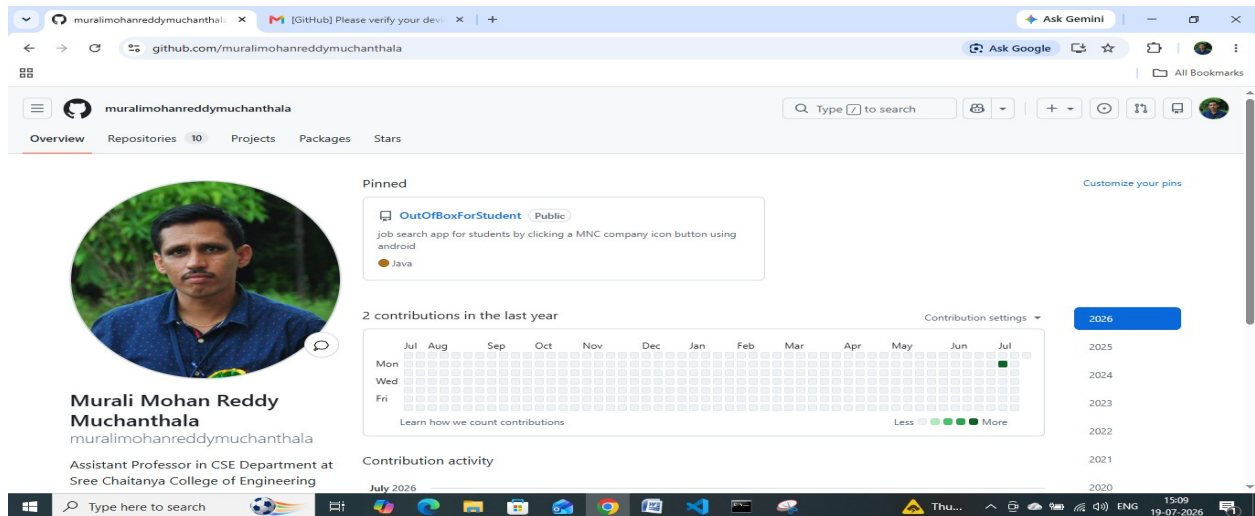
Step 2 Create GitHub Account

Visit GitHub

Create account

Verify email

Login

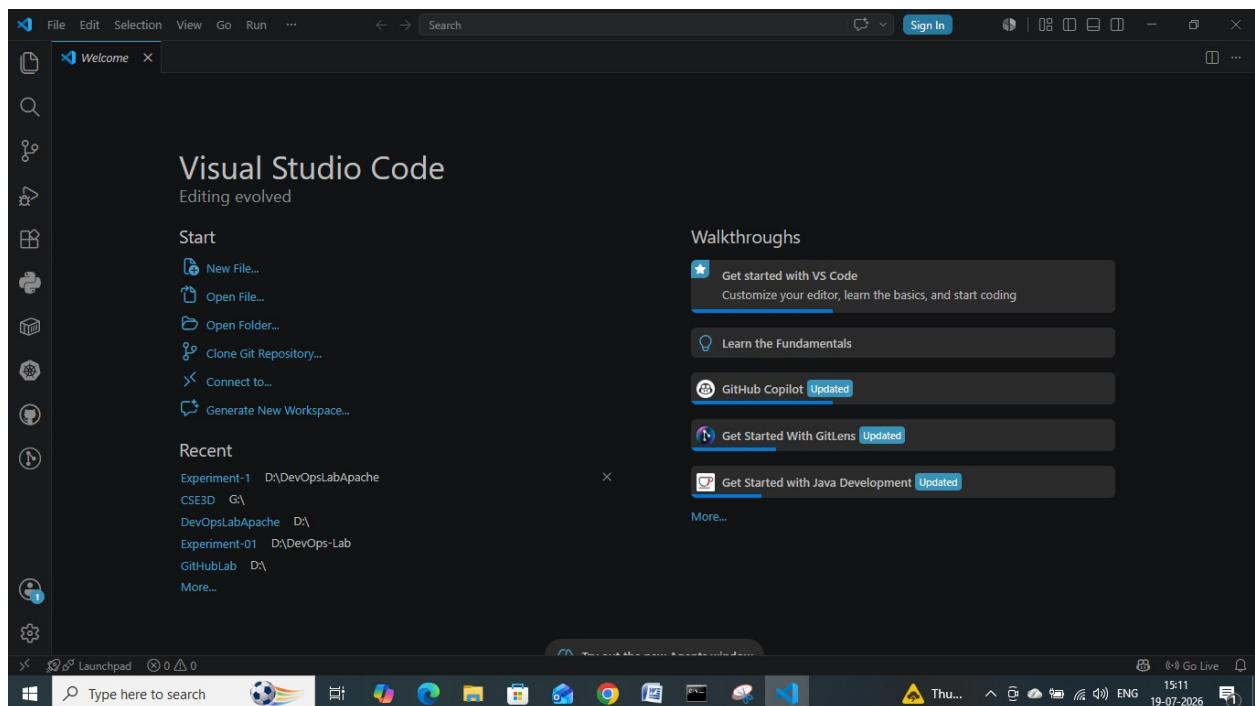


Step 3 Install VS Code

Download VS Code

Install

Open VS Code



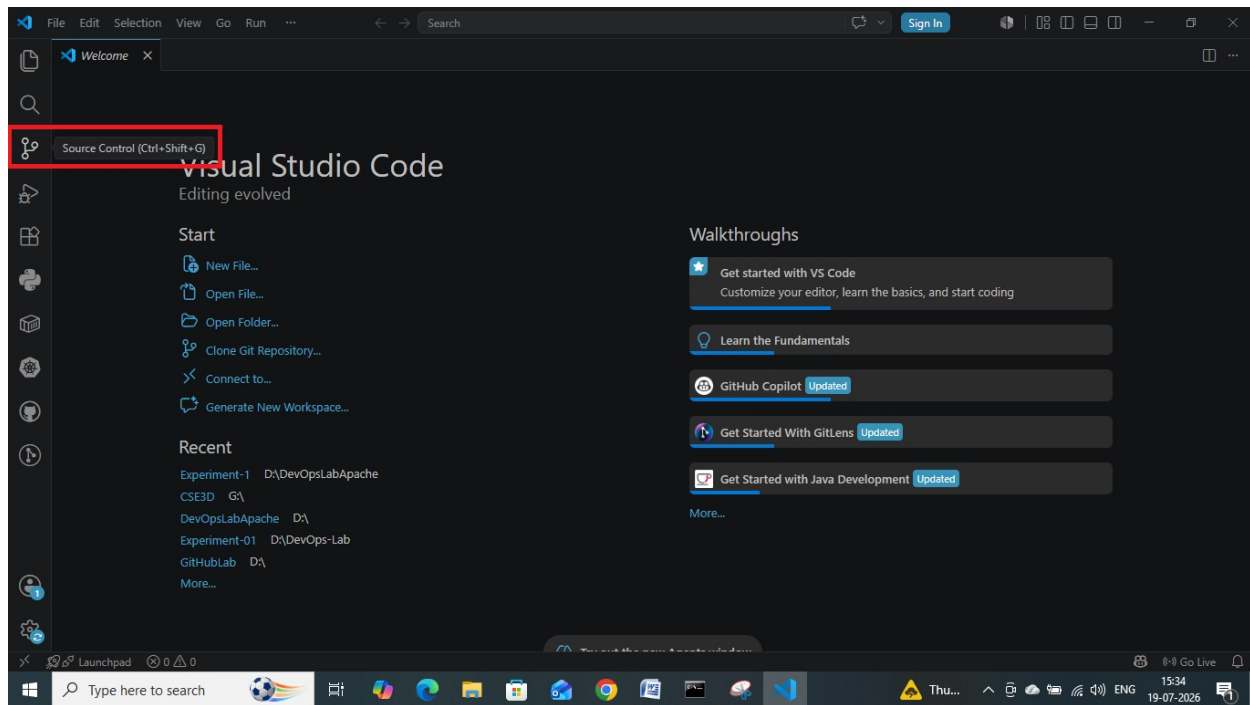
Step 4 Install Git Extension in VS Code

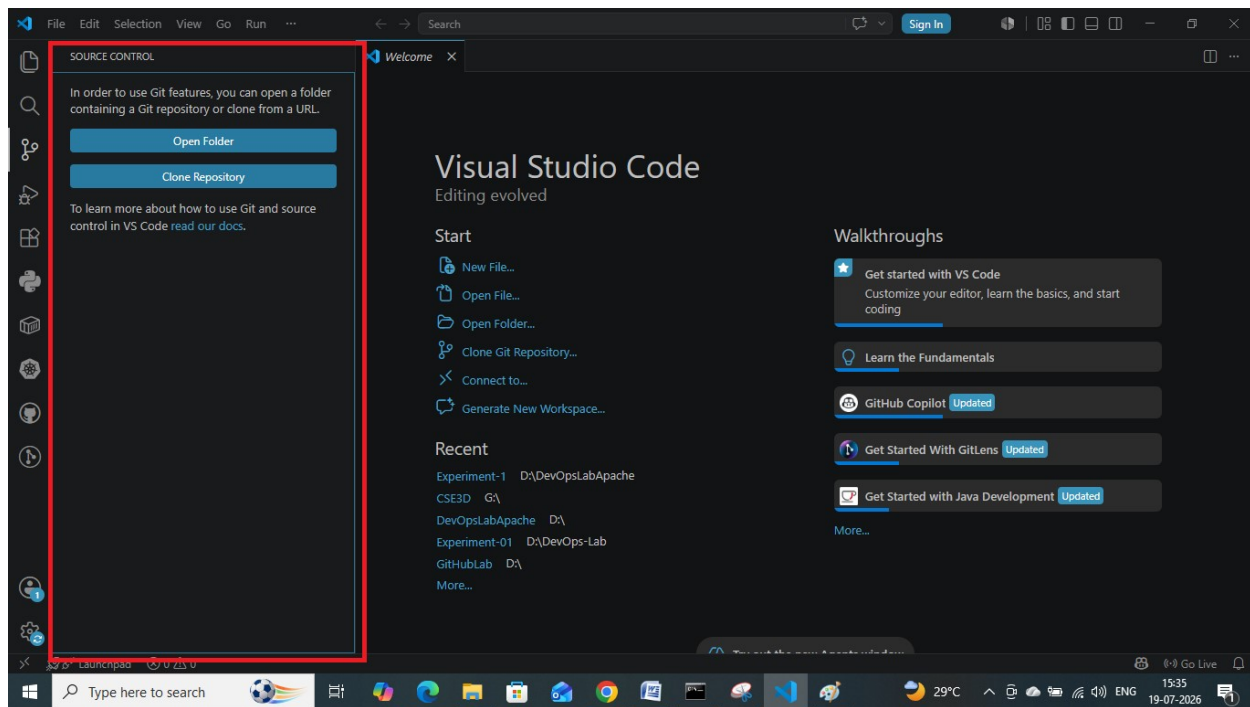
Usually built-in.

Check

Source Control Icon

on left sidebar.





6. Configure Git

Open Git Bash

Run

```
git config --global user.name "Your Name"
```

Example

```
git config --global user.name "Rahul"
```

Now configure email

```
git config --global user.email "abc@gmail.com"
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Admin> git --version
git version 2.55.0.windows.2
PS C:\Users\Admin> git config --global user.name "Murali Mohan Reddy Muchanthala"
PS C:\Users\Admin> git config --global user.email "mcuberreddy@gmail.com"
PS C:\Users\Admin>
```

Check configuration

```
git config --list
```

```
PS C:\Users\Admin> git config --list

diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
:
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
```



```
core.symlinks=false
```

```
pull.rebase=false
```

```
credential.helper=manager
```

```
credential.https://dev.azure.com.usehttppath=true
```

```
init.defaultbranch=master
```

```
core.editor="C:\Users\Admin\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait
```

```
user.name=Murali Mohan Reddy Muchanthala
```

```
user.email=mcuberreddy@gmail.com
```

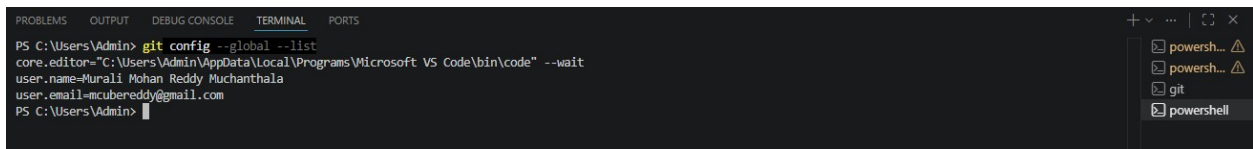
```
~
```

```
~
```

Output

```
user.name=Rahul
```

```
user.email=abc@gmail.com
```



```
PS C:\Users\Admin> git config --global --list
core.editor="C:\Users\Admin\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait
user.name=Murali Mohan Reddy Muchanthala
user.email=mcuberreddy@gmail.com
PS C:\Users\Admin>
```

7. Creating First Repository

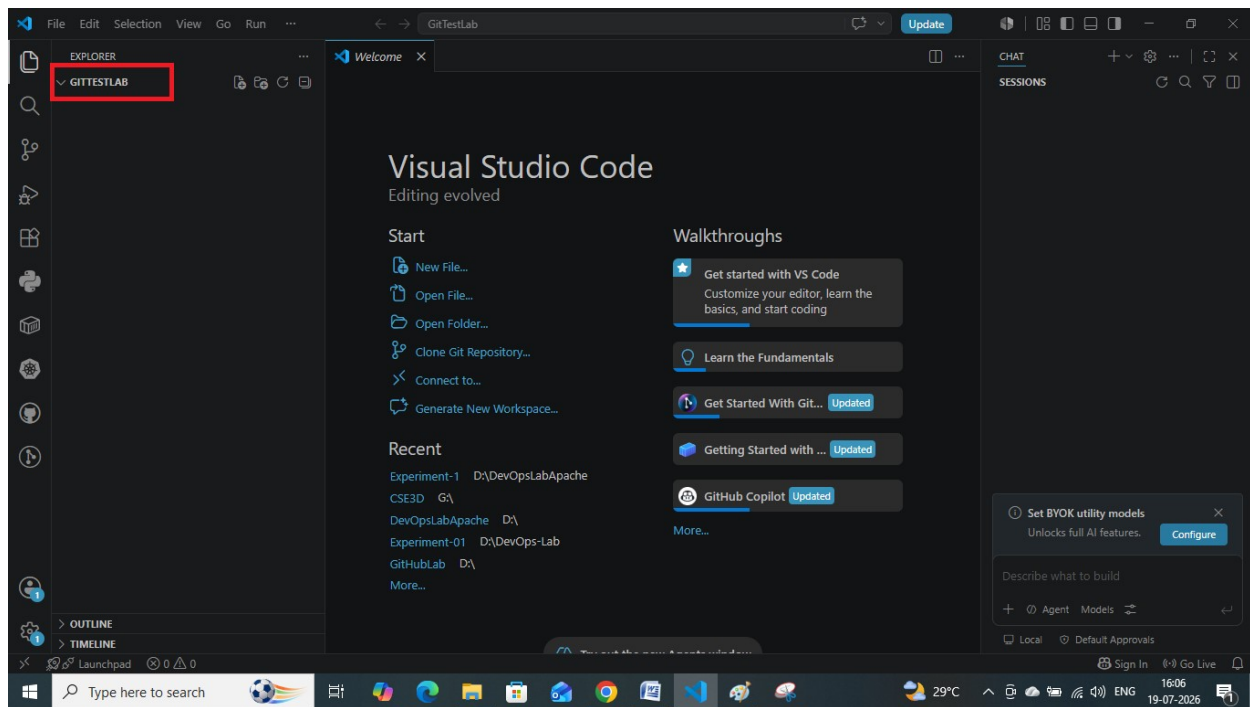
Create folder

GitTestLab



19-07-2026 16:03

File folder



Open Git Bash inside folder

Initialize repository

```
git init
```

Output

Initialized empty Git repository

Now folder becomes Git Repository.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [X] [Y] [Z] X
PS D:\GitTestLab> git init
Initialized empty Git repository in D:/GitTestLab/.git/
PS D:\GitTestLab> |
```

8. Understanding Git Repository

Repository stores

- Files

- Versions
- Commits
- History

Inside folder hidden directory exists

.git



It contains complete Git database.

9. Create First File

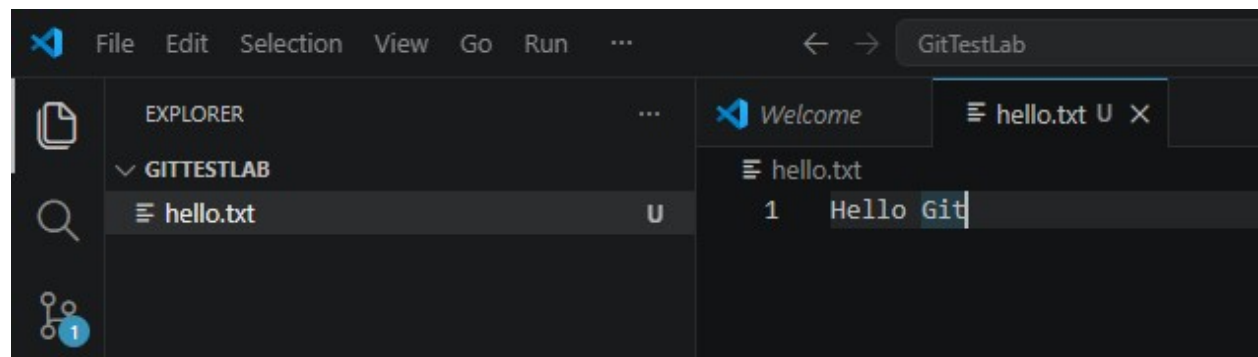
Create

hello.txt

Write

Hello Git

Save.



10. Check Repository Status

Command

```
git status
```

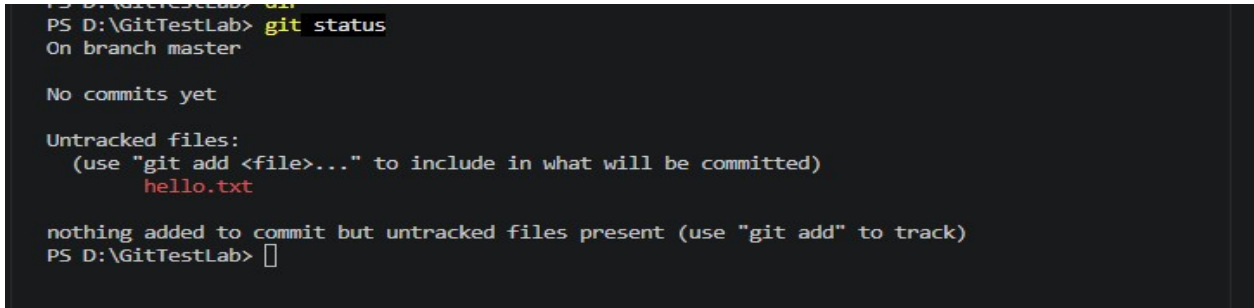
Output

```
Untracked file:  
hello.txt
```

Meaning

Git sees file

But not tracking it.

A terminal window with a dark background. The prompt is 'PS D:\GitTestLab>'. The command 'git status' has been entered. The output shows 'On branch master', 'No commits yet', and 'Untracked files: (use "git add <file>..." to include in what will be committed) hello.txt'. A final line says 'nothing added to commit but untracked files present (use "git add" to track)'. The prompt 'PS D:\GitTestLab>' is shown again with a cursor.

```
PS D:\GitTestLab> git status  
On branch master  
  
No commits yet  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
      hello.txt  
  
nothing added to commit but untracked files present (use "git add" to track)  
PS D:\GitTestLab> █
```

11. Add File

Command

```
git add hello.txt
```

Or

```
git add .
```

Meaning

Move file to staging area.

A terminal window with a dark background. The prompt is 'PS D:\GitTestLab>'. The command 'git add hello.txt' has been entered. The prompt 'PS D:\GitTestLab>' is shown again with a cursor.

```
PS D:\GitTestLab> git add hello.txt  
PS D:\GitTestLab> █
```

12. Check Status Again

```
git status
```

Output

Changes to be committed

Now Git is tracking file.

```
PS D:\GitTestLab> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.txt

PS D:\GitTestLab> █
```

13. Commit Changes

Commit saves snapshot.

Command

```
git commit -m "First Commit"
```

Output

1 file changed

```
PS D:\GitTestLab> git commit -m "First Commit"
[master (root-commit) 482e73f] First Commit
1 file changed, 1 insertion(+)
create mode 100644 hello.txt
PS D:\GitTestLab> █
```

14. View Commit History

`git log`

Output

commit
Author
Date
Message

```
PS D:\GitTestLab> git log
commit 482e73f2bf19fb259daf685c259e0a6b31e41cfa (HEAD -> master)
Author: Murali Mohan Reddy Muchanthala <mcuberreddy@gmail.com>
Date: Sun Jul 19 16:22:29 2026 +0530
```

```
First Commit
PS D:\GitTestLab> █
```

Short history

`git log --oneline`

Example

fd4567 First Commit

```
PS D:\GitTestLab> git log --oneline
482e73f (HEAD -> master) First Commit
PS D:\GitTestLab> █
```

15. Modify File

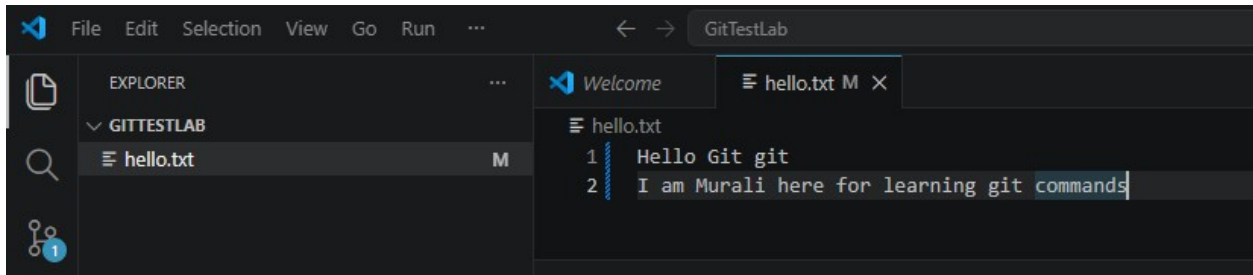
Open

hello.txt

Add

Learning Git Commands

Save



Check status

```
git status
```

Output

Modified

```
PS D:\GitTestLab> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.txt

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\GitTestLab>
```

16. Commit Again

```
git add .
```

```
PS D:\GitTestLab> git add .
PS D:\GitTestLab>
```

```
git commit -m "Updated File"
```

```
PS D:\GitTestLab> git commit -m "Updated File"
[master 74bc8d3] Updated File
 1 file changed, 2 insertions(+), 1 deletion(-)
PS D:\GitTestLab>
```

History

```
git log --oneline
```

```
PS D:\GitTestLab> git log --oneline
74bc8d3 (HEAD -> master) Updated File
482e73f First Commit
PS D:\GitTestLab>
```

Output

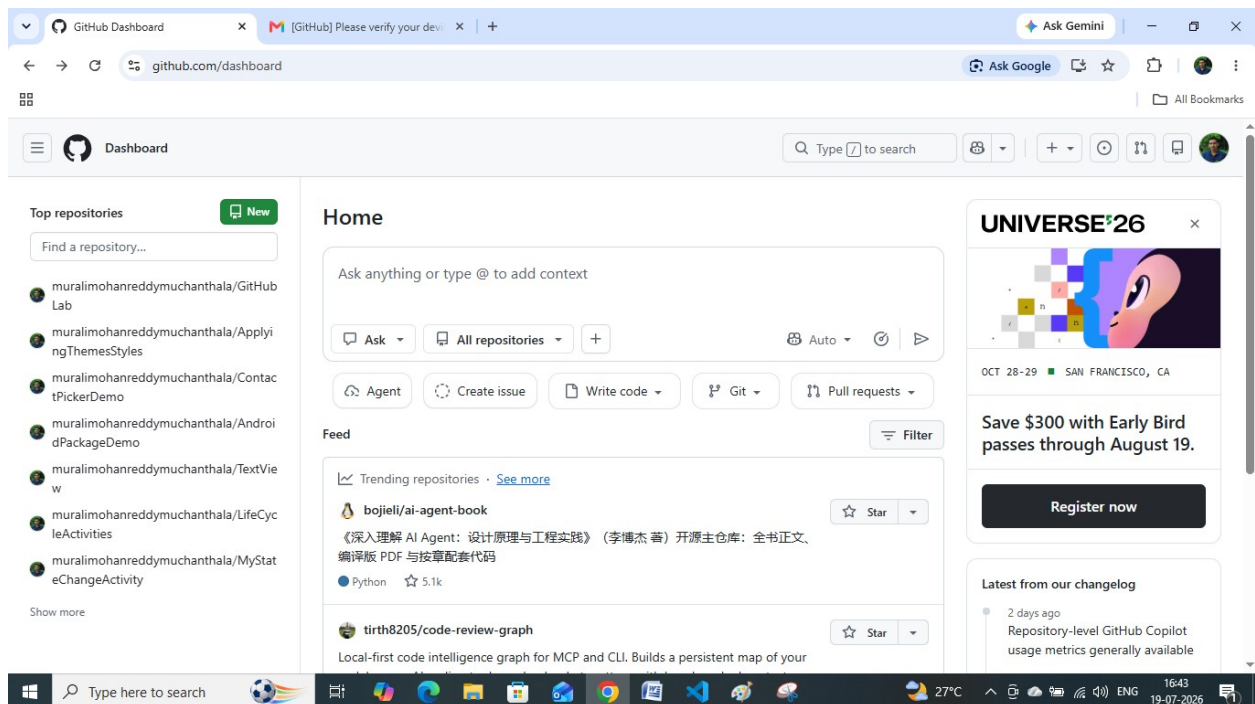
Update File

First Commit

```
PS D:\GitTestLab> git log --oneline
74bc8d3 (HEAD -> master) Updated File
482e73f First Commit
PS D:\GitTestLab> |
```

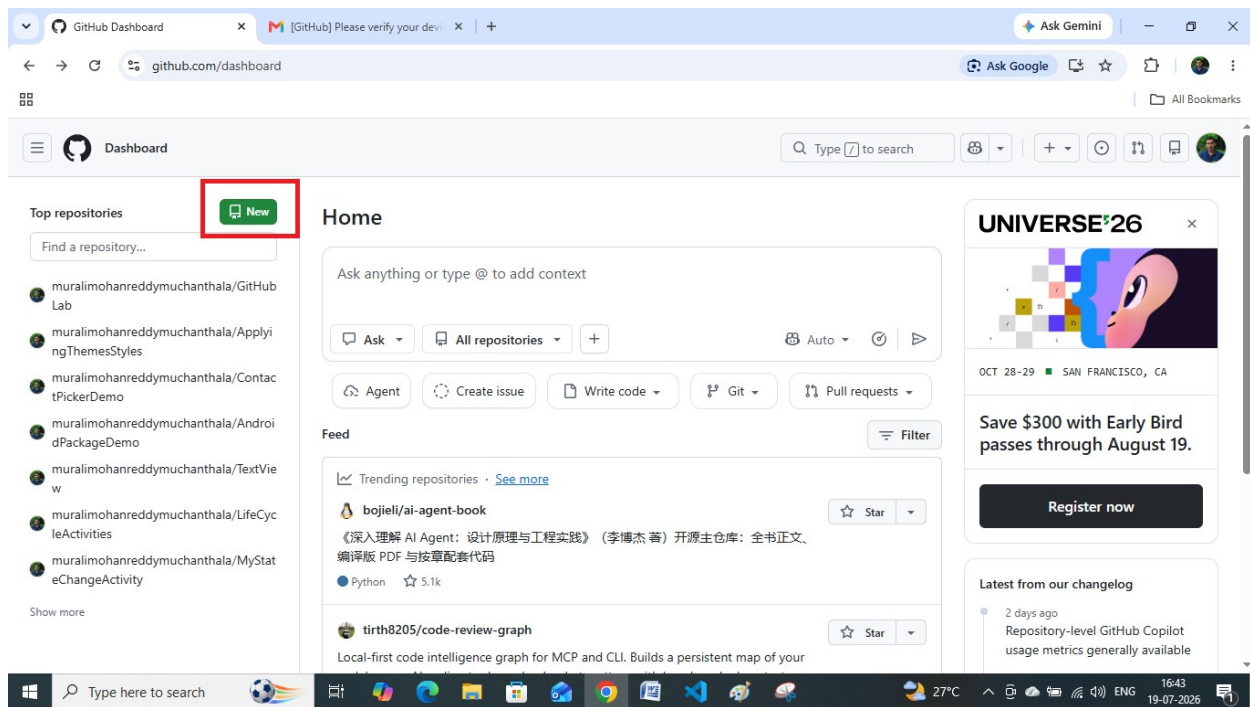
17. Create GitHub Repository

Login GitHub



Click

New Repository

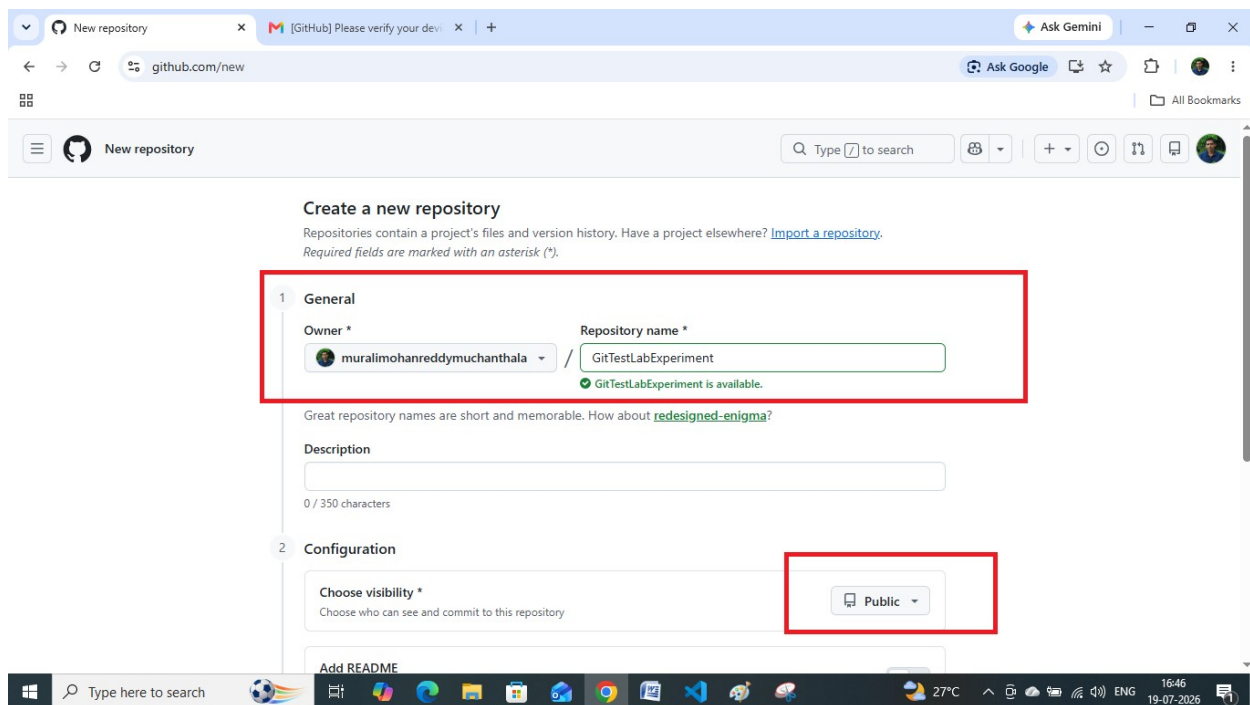


Example

GitTestLabExperiment

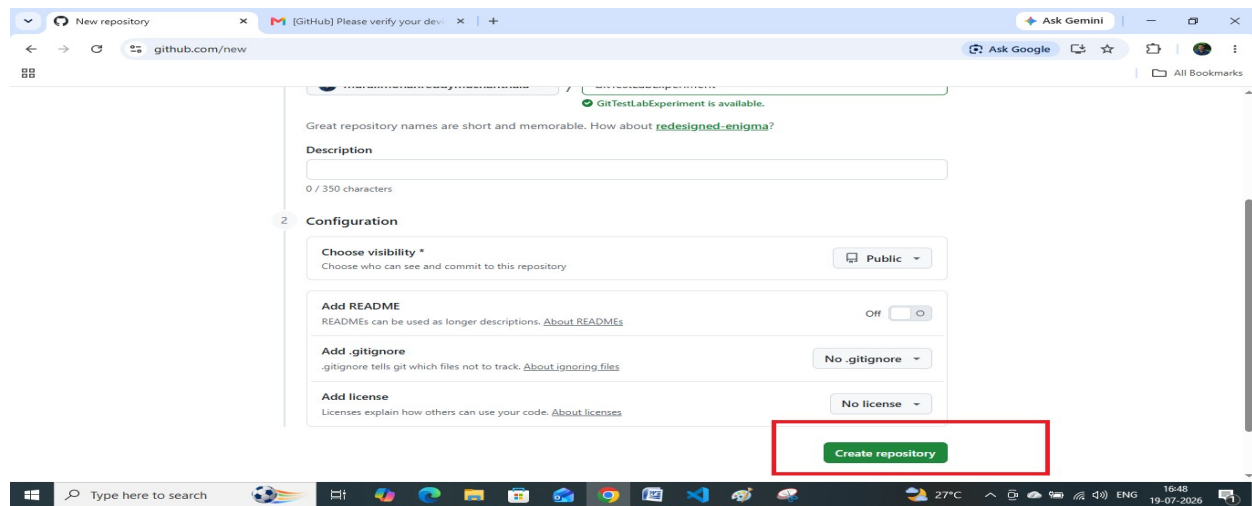
Keep

Public



Click

Create Repository

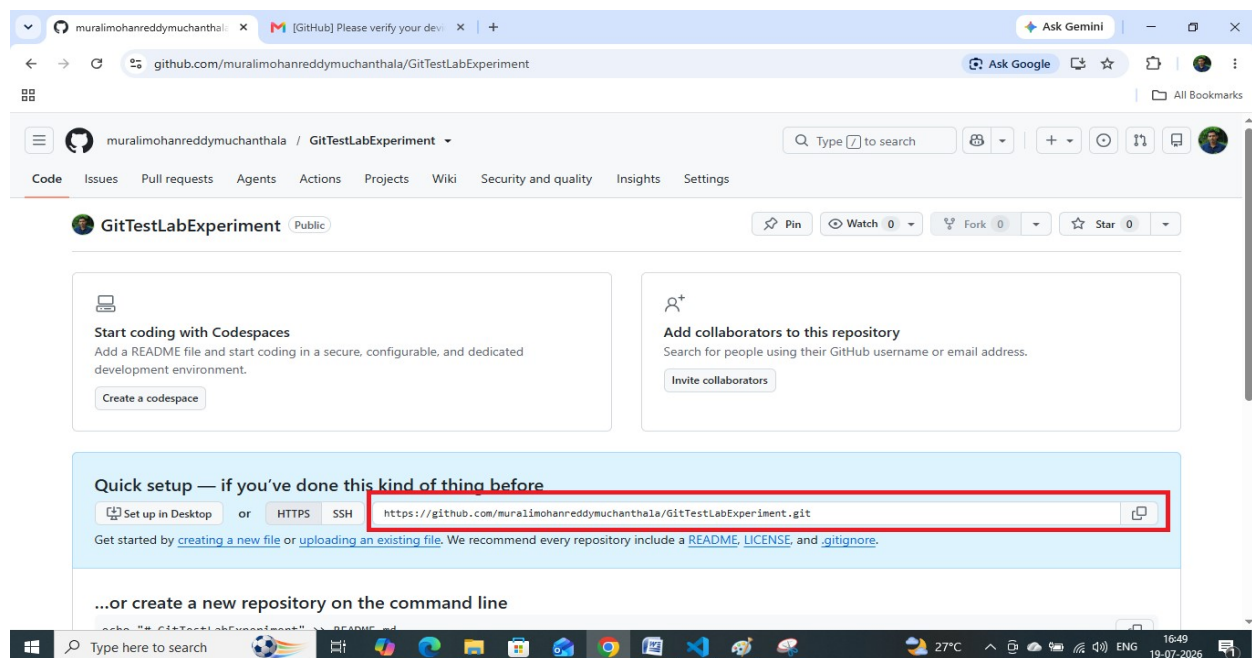


18. Connect Local Repository to GitHub

Copy repository URL

Example

`https://github.com/username/GitTestLabExperiment.git`



<https://github.com/muralimohanreddymuchanthala/GitTestLabExperiment.git>

Run

```
git remote add origin  
https://github.com/muralimohanreddymuchanthala/GitTestLabExperiment.git
```

```
PS D:\GitTestLab> git remote add origin https://github.com/muralimohanreddymuchanthala/GitTestLab  
Experiment.git  
PS D:\GitTestLab> 
```

Check

```
git remote -v
```

Output

Origin

```
PS D:\GitTestLab> git remote -v  
origin https://github.com/muralimohanreddymuchanthala/GitTestLabExperiment.git (fetch)  
origin https://github.com/muralimohanreddymuchanthala/GitTestLabExperiment.git (push)  
PS D:\GitTestLab> 
```

19. Push Code to GitHub

Rename branch

```
git branch -M main
```

```
PS D:\GitTestLab> git branch -M main  
PS D:\GitTestLab> 
```

Push

```
git push -u origin main
```

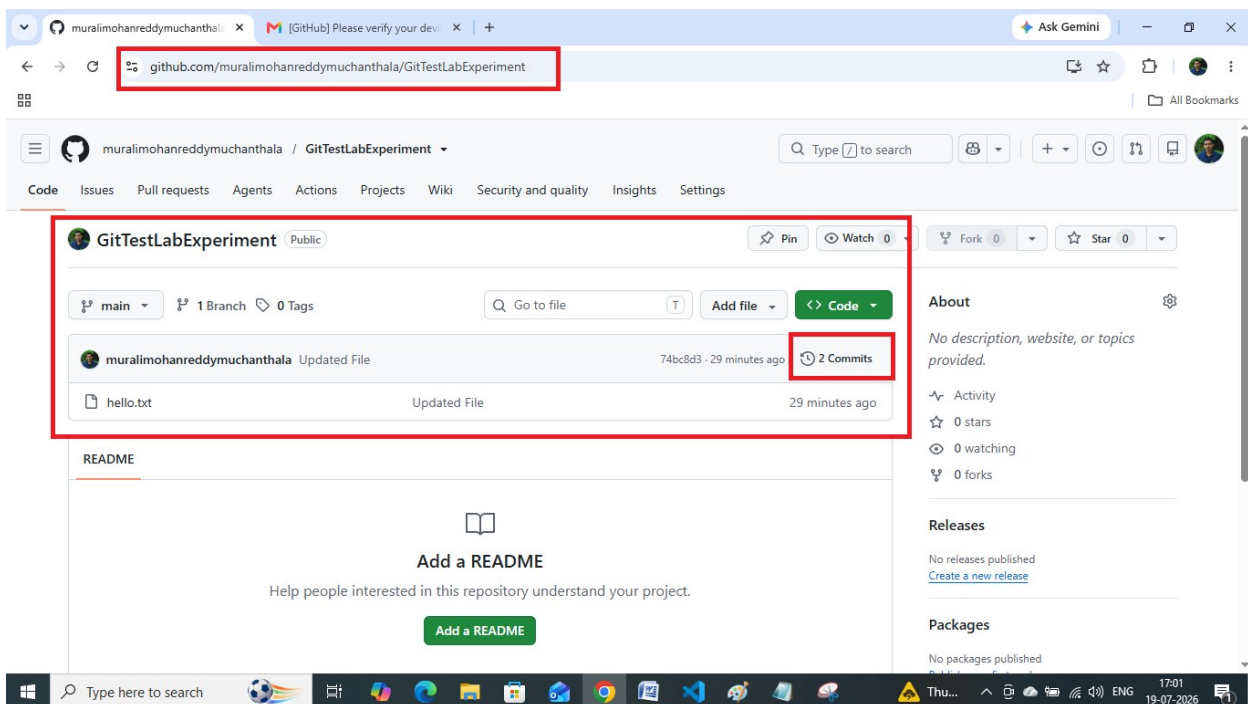
Output

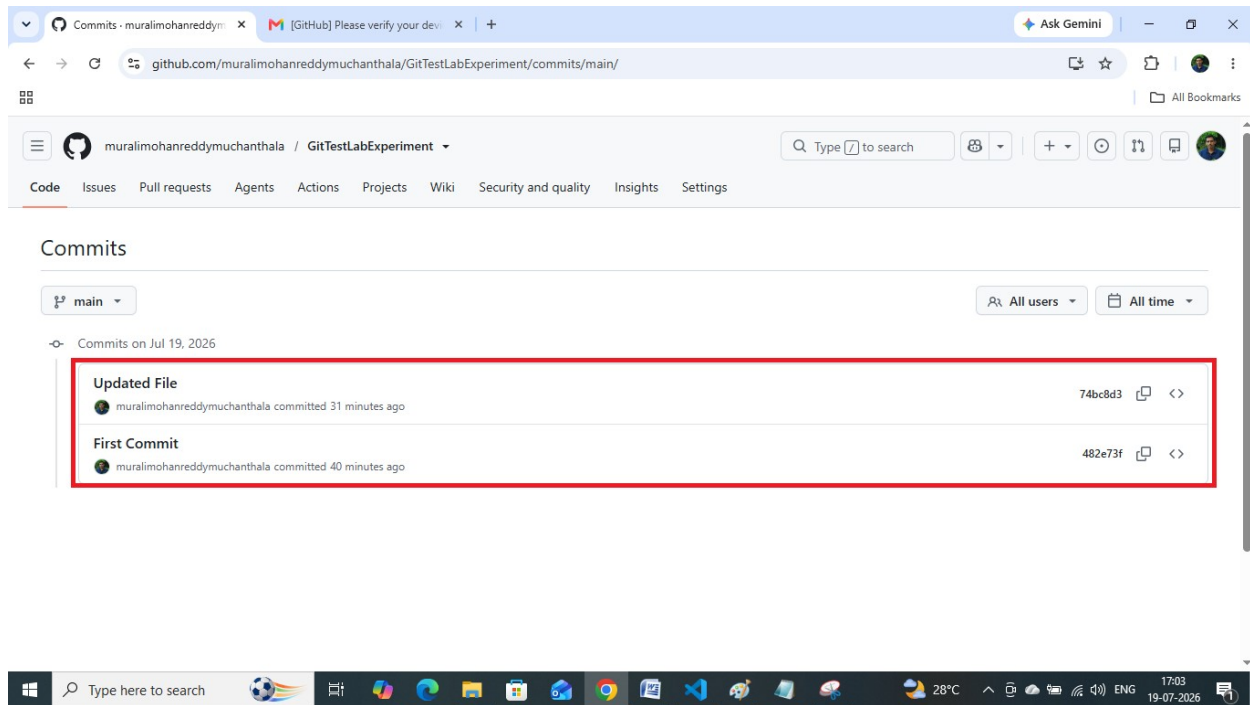
```
Writing objects...  
Done
```

```
PS D:\GitTestLab> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 476 bytes | 68.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/muralimohanreddymuchanthala/GitTestLabExperiment.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS D:\GitTestLab>
```

Refresh GitHub.

Files appear online.





20. Clone Repository

Command

```
git clone https://github.com/username/GitLabExperiment.git
```

Output

Cloning into...

Repository downloaded.

```
PS D:\GitTestLab> git clone https://github.com/muralimohanreddymuchanthala/GitTestLabExperiment.git
Cloning into 'GitTestLabExperiment'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 6 (delta 1), reused 6 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
Resolving deltas: 100% (1/1), done.
PS D:\GitTestLab>
```

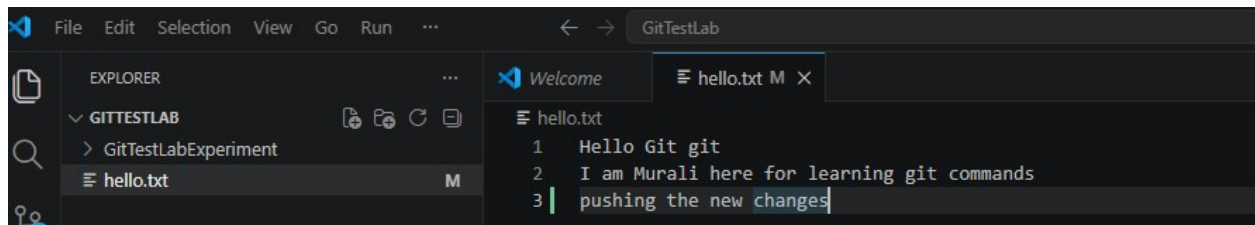

21. Pull Latest Changes

```
git pull
```

Downloads latest changes from GitHub.

```
PS D:\GitTestLab> git pull
Already up to date.
PS D:\GitTestLab> 
```

22. Push New Changes



```
git add .
```

```
PS D:\GitTestLab> git add .
warning: adding embedded git repository: GitTestLabExperiment
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> GitTestLabExperiment
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached GitTestLabExperiment
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
PS D:\GitTestLab> 
```

```
git commit -m "New Update"
```

```
PS D:\GitTestLab> git commit -m "New Update"
[main 24672d2] New Update
 2 files changed, 3 insertions(+), 1 deletion(-)
 create mode 160000 GitTestLabExperiment
PS D:\GitTestLab>
```

```
git push
```

```
PS D:\GitTestLab> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 378 bytes | 94.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/muralimohanreddymuchanthala/GitTestLabExperiment.git
 74bc8d3..24672d2  main -> main
PS D:\GitTestLab>
```

23. Branch Commands

Create Branch

```
git branch feature
```

```
PS D:\GitTestLab> git branch feature
PS D:\GitTestLab>
```

View Branches

```
git branch
```

```
PS D:\GitTestLab> git branch
feature
* main
PS D:\GitTestLab>
```

Switch Branch

```
git checkout feature
```

Or

```
git switch feature
```

```
PS D:\GitTestLab> git checkout feature
Switched to branch 'feature'
PS D:\GitTestLab> git switch feature
Already on 'feature'
PS D:\GitTestLab> |
```

24. Merge Branch

Go to main

```
git checkout main
```

```
PS D:\GitTestLab> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS D:\GitTestLab> |
```

Merge

```
git merge feature
```

```
PS D:\GitTestLab> git merge feature
Already up to date.
PS D:\GitTestLab> |
```

25. Delete Branch

```
git branch -d feature
```

```
PS D:\GitTestLab> git branch -d feature
Deleted branch feature (was 24672d2).
PS D:\GitTestLab> |
```

26. Git Commands Summary

Command	Purpose
git --version	Check Git installation
git config --global user.name	Set username
git config --global user.email	Set email

Command	Purpose
git config --list	View configuration
git init	Initialize repository
git status	Check repository status
git add filename	Stage file
git add .	Stage all files
git commit -m "msg"	Save changes
git log	View history
git log --oneline	Short history
git remote -v	View remote repository
git push	Upload changes
git pull	Download changes
git clone URL	Copy repository
git branch	List branches
git branch name	Create branch
git switch name	Switch branch
git merge name	Merge branch
git branch -d name	Delete branch

27. Implementation Using VS Code

Step 1

Open VS Code.

Step 2

Open your project folder.

Step 3

Open **Terminal** → **New Terminal**.

Step 4

Initialize Git.

```
git init
```

Step 5

Create a file.

Example

```
index.html
```

Step 6

Click the **Source Control** icon (left sidebar).

You will see:

```
Changes  
index.html
```

Step 7

Click + to stage the file (equivalent to `git add`).

Step 8

Type a commit message in the message box.

Example:

```
Initial Commit
```

Click **Commit (✓)** or run:

```
git commit -m "Initial Commit"
```

Step 9

Publish the repository or use:

```
git remote add origin <repository-url>  
git push -u origin main
```

Step 10

Verify that the repository and committed files appear on GitHub.

28. Expected Output

- Git installed successfully.
 - GitHub account created.
 - Local repository initialized.
 - Files tracked by Git.
 - Commits created successfully.
 - Repository connected to GitHub.
 - Code pushed to GitHub.
 - Repository cloned successfully.
 - Git operations performed successfully from VS Code.
-

29. Viva Questions

1. What is Version Control System (VCS)?
 2. What is Git?
 3. What is GitHub?
 4. Differentiate Git and GitHub.
 5. What is a Git repository?
 6. What is the purpose of the `.git` folder?
 7. Explain the Git workflow (Working Directory → Staging Area → Repository).
 8. What is the purpose of `git init`?
 9. What is the difference between `git add` and `git commit`?
 10. What is the purpose of `git status`?
 11. What is a branch in Git?
 12. What is the difference between `git pull` and `git push`?
 13. What is the purpose of `git clone`?
 14. What is a commit message?
 15. How does VS Code integrate with Git?
-

30. Conclusion

In this experiment, we learned the fundamentals of **Version Control** using **Git** and **GitHub**. We configured Git, created a local repository, tracked files, committed changes, connected the repository to GitHub, pushed and pulled code, cloned repositories, managed branches, and performed the same operations using **VS Code**. These are essential DevOps skills used in software development and collaborative projects.